FIN-RSI: An Agent-Native Library of Financial Skills, Checks, and Self-Improvement

Anonymous submission

Abstract

Financial research code fails silently: a program that applies stock splits twice, or trades on the closing price it reads, still runs and still returns a backtest. We present FIN-RSI, an agent-native Python library that puts three things behind interfaces a language-model agent can call. *Skills* are 131 dated skill documents, 48 algorithms and 6 rule cards. *Checks* are 39 guards, including property checks that need no expected output. *Self-improvement* is 7 methods, from self-debugging and reflection to playbook learning and prompt evolution, behind one `fit`/`solve`/`state` interface with metered, equal budgets. With the library comes a suite of 34 financial programming tasks. Each task has an independent reference and a hidden grader that no method sees, and the suite is split by algorithm into development and held-out sets. The evaluation protocol separates the contributions of skills, checks and self-improvement. [One sentence of results once the experiments are complete.] The library, the tasks and the protocol will be released.

1 Introduction

Financial research code fails silently. Some programs apply stock splits twice. Others decide a position with the closing price it trades over, or leave warm-up rows missing. Each of these programs still runs and still returns a backtest, and nothing in the output says it is wrong. Language-model agents now write such code for trading and financial research (Zhang et al. 2024; Bigeard et al. 2025), and they make these mistakes. In our earlier audit, a 14B coding model wrote 24 programs. Every one re-applied split factors that the prompt defined as already cumulative, and every one failed a split check.

Three things would help an agent avoid them:

- knowledge of the failure modes;
- executable checks that detect each failure without an expected output;
- a way to turn the results of those checks into better programs.

All three exist today, but separately. Skill documents and tool collections give agents knowledge. Testing utilities give checks. Self-improvement methods each come with their own tasks, feedback and budgets (Shinn et al. 2023; Chen et al. 2024; Zhao et al. 2024; Agrawal et al. 2026). As a result, a practitioner cannot tell which self-improvement structure helps on financial code. Nor can they tell whether the knowledge and checks help at all.

We present FIN-RSI, a Python library that puts all three behind interfaces an agent can call (Figure 1). We follow PyOD, which put outlier detectors behind one interface (Zhao et al. 2019), and ADBench, which compared them under equal conditions (Han et al. 2022). In the same way, FIN-RSI places self-improvement methods behind one `fit` / `solve` / `state` interface, and gives every method the same judge and the same metered budget. A user connects a model once, as one `generate(messages)` callable, and every part of the library uses it.

Our contributions:

- **An agent-native library** (§3): 131 dated skill documents; 48 algorithms; 6 rule cards; and 39 guards, exported as 67 tools for MCP, OpenAI and Anthropic clients.
- **Self-improvement behind one interface** (§4): 7 methods with equal budgets. They range from self-debugging and reflection to a playbook loop that accepts an edit only when it does not break a task that already passed.
- **A task suite and protocol** (§5): 34 financial programming tasks, each with an independent reference and a hidden grader that no method sees. The tasks are split by algorithm into development and held-out sets. An ablation over 4 skill arms separates the contributions of skills, checks and self-improvement.

[Main findings, one or two sentences, once the experiments are complete.]

2 Related work

Self-improvement without weight updates. Several methods let a frozen model improve its own output.

- In Self-Refine, the same model critiques its output and revises it (Madaan et al. 2023). Huang et al. (2024) find that this kind of intrinsic self-correction, with no external signal, rarely helps reasoning and can make it worse.

This is an anonymized submission for review purposes only. Distribution, citation, or public sharing of this manuscript is strictly prohibited. Copyright and publication details will appear in the final version if accepted.

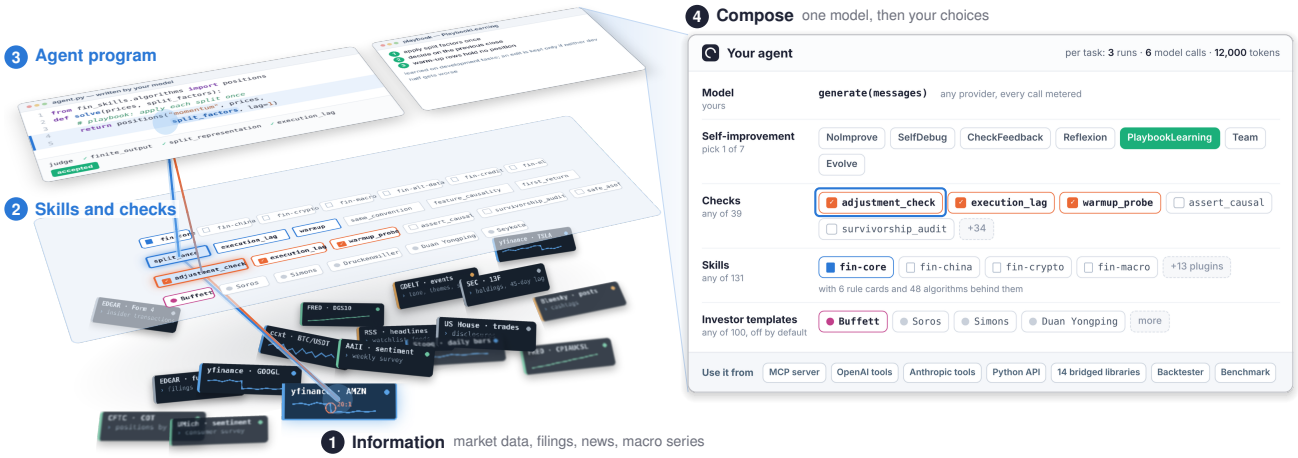


Figure 1: Fin-RSI at a glance. (1) Information comes in from market data, filings, news and macro series. (2) The user picks skills, rule cards, checks and investor templates, and (3) the model writes the agent's program under one self-improvement method. The thread follows one real trap, Amazon's 20-for-1 split in June 2022: the `split_once` rule card and `adjustment_check` cover it, and the program passes the split factors to `positions()`; the program shown is one that passes every check. (4) The same agent laid flat, with the interfaces it can be used from.

- Methods that bring in an external signal do better. CRITIC consults tools such as search engines and interpreters (Gou et al. 2024), and Self-Debugging uses execution results and unit tests (Chen et al. 2024).
- Other methods carry what was learned across attempts or tasks. Reflexion keeps verbal reflections (Shinn et al. 2023). ExpeL extracts insights from training tasks (Zhao et al. 2024). Dynamic Cheatsheet curates a memory at test time (Suzgun et al. 2026). GEPA evolves prompts by reflecting on trajectories (Agrawal et al. 2026).

Each of these was evaluated with its own tasks, feedback and budget, so their results cannot be compared directly. Fin-RSI places them behind one interface with one judge and one budget, so that differences come from the method. Its checks supply the external signal that Huang et al. found necessary.

Libraries that ship with a benchmark. PyOD made outlier detection a matter of choosing a class (Zhao et al. 2019), and ADBench used that shared interface to compare detectors under equal conditions (Han et al. 2022). We follow that pattern for agent self-improvement. Voyager also keeps a library of executable skills, but these are programs the agent wrote for itself (Wang et al. 2024). The skills in Fin-RSI are written ahead of time and dated, and each claim is marked by how it was verified; the rule cards and checks in the library are tied to those skills.

Financial LLMs and agents. FinBen evaluates language models on a broad set of financial tasks (Xie et al. 2024). FinAgent combines market information with tools, reflection and memory for trading (Zhang et al. 2024). StockBench evaluates sequential portfolio decisions (Chen et al. 2025). The Finance Agent Benchmark evaluates financial research tasks (Bigeard et al. 2025). These benchmarks score answers or returns. We score whether a research program is correct against a reference: a backtest return can reward a wrong program on a favourable price path, and repeated trials make such false discoveries likely (López de Prado 2018; Bailey et al. 2017).

Testing without expected outputs. Metamorphic testing checks a relation between the outputs on related inputs when the expected output itself is unknown (Segura et al. 2016). CheckList brought behavioural tests of this kind to NLP models (Ribeiro et al. 2020). Two of our checks are metamorphic relations for financial research code:

- the same economic prices, written two ways, must give the same positions;
- changing future prices must not move past positions.

They catch the corporate-action and look-ahead errors that a backtest does not reveal.

3 The Fin-RSI library

Fin-RSI is a Python package whose base install depends only on numpy, pandas and scipy. It has three layers. *Skills* hold the financial knowledge a model needs and the algorithms that apply it. *Checks* are executable tests for the failures that the knowledge is meant to prevent. *Self-improvement* methods turn check results into better programs (§4). An agent reaches all three through the same interfaces (§3.3; Figure 1).

3.1 Skills: documents, algorithms and rule cards

Skill documents. The library ships 131 skills in 17 plugins that follow the Agent Skills specification (Agent Skills 2026), with 75 reference files and 125 runnable scripts. A skill is not a tutorial. It records the defaults, licence changes and silent failure modes that a model's training prior gets wrong. Each claim is dated and marked by how it was verified: at a primary source, secondhand, silently wrong, or dead. The descriptions of all 131 skills take about 27,700 tokens of an agent's skill listing, so the skills are split into

Card	Verified by	Call
timing		
execution_lag	execution_lag	yes
feature_causality	assert_causal	–
corporate actions		
split_once	split_representation adjustment_check	yes
same_convention	split_representation	–
data hygiene		
warmup	finite_output	–
first_return	finite_output	–

Table 1: Rule cards by family, the checks that verify each one, and whether the card names a library call that follows the rule.

plugins and a user installs only the ones a task needs. The plugin marketplace also lists 92 federated packs from other repositories (2,390 skills), off by default and not verified here. 3 of them are investor templates: 100 skills built on named investors' approaches, such as Buffett's, Soros's or Duan Yongping's. The `kol-credibility-registry` skill adds track records for 40 social-media authors.

Algorithms. A catalog of 48 algorithm entries covers 11 task types, including portfolio construction, trading signals, volatility, risk, forecasting, pricing and execution. 25 are implemented in the library. The other 23 wrap 13 optional libraries; when a library is absent, its entries are reported as missing dependencies rather than replaced silently. Two functions turn one algorithm into the object a research program needs:

```
w = positions("macd", raw, factors,
              parameters={"fast": 8})
v = rolling("ewma_volatility", raw,
            factors, window=40)
```

`positions()` takes raw quotes and split factors, applies the factors once, and returns positions decided on the previous close. `rolling()` does the same for volatility, risk and forecast algorithms, with one lagged value per date and asset. `adjust_prices()` marks its output as adjusted, and `positions()` refuses to adjust such a frame again, because passing adjusted prices together with their factors applies every split twice.

Rule cards. Between the documents and the code sit 6 rule cards in 3 families: timing, corporate actions and data hygiene (Table 1). A card holds:

- a few sentences of rule text;
- the skill passage the rule comes from;
- the checks that verify it;
- the task traits that select it;
- where one exists, the library call that follows the rule.

A task declares its traits, such as *positions* or *split factors*, and receives only the cards those traits select. A run without library access receives the same cards without the library calls.

3.2 Checks

The library has 39 guards behind one interface: `run(**inputs)` returns an outcome with `passed`, a message and warnings. A failing guard is a result, not an error. Malformed arguments are reported as tool errors that name the field, so a model can correct its call. 32 guards take JSON inputs and are exported as tools. The other 7 need a live Python object, such as the position function under test. A tool schema cannot carry one, and a stand-in built by the model would not be the caller's function, so these guards are available only in Python.

For generated programs, the library provides property checks that need no expected output. Three are used throughout this paper:

- `finite_output`: the result has no missing or infinite values.
- `split_representation`: the program is re-run on the same economic prices, given as adjusted closes with unit factors, and its output must not change (absolute tolerance 10^{-8}). A program that applies the factors twice, or mixes raw and adjusted prices, fails.
- `execution_lag`: the prices from row k onward are perturbed, and no position up to row k may move. A rule that trades on the close it reads fails this check. It still passes a plain causality test, which lets the value on row t use row t.

Each check carries a *cause*: one sentence saying what the check observed, for instance that a position moved when only prices on or after its own row changed. `explain()` pairs each cause with the cards that address it. We treat the cause as part of the check and the card as part of the skills; §4.1 separates the two.

3.3 Agent interfaces

One registry yields 67 tools under the same names in three formats: a Model Context Protocol server over stdio (Model Context Protocol 2026), OpenAI tool schemas and Anthropic tool schemas. The tools read and search skills; list, recommend and run algorithms; and run the exported guards. Python code imports the same functions directly.

`agents.Team` turns one user model, a callable `generate(messages) -> str`, into a writer and one reviewer per rule family. Code checks decide acceptance, and no role can accept a program the checks reject. The self-improvement methods take the same `generate` callable, so a user connects one model to every part of the library.

4 Self-improvement behind one interface

PyOD put outlier detectors behind one `fit` / `decision_function` interface (Zhao et al. 2019). That interface later let ADBench compare 30 detectors on equal terms (Han et al. 2022). FIN-RSI does the same for self-improvement. Every method is a class with one constructor and three calls:

```
m = rsi.Reflexion(library=True,
                  cards=True)
m.fit(generate, tasks.dev)
```

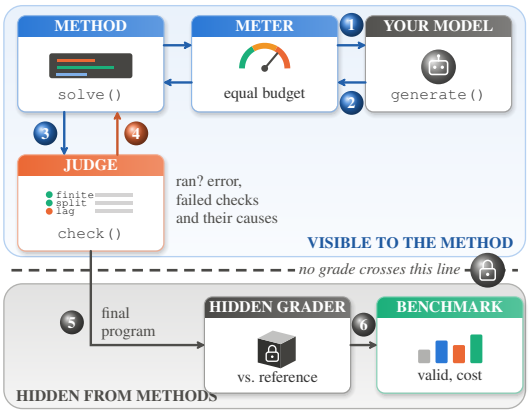


Figure 2: One episode. (1) Each prompt and (2) each program pass through the meter; (3) the method sends code to the judge and (4) sees only what `check()` returns. (5) After `solve()` returns, the hidden grader runs on the final program, and (6) its verdict goes only to the benchmark report.

```
ep = m.solve(generate, task, seed=11)
ep.accepted, ep.code, ep.usage
m.state()
```

`generate` is the user's own model. `fit()` does nothing for a method that does not learn. `state()` returns what the method learned, which is hashed into every episode record. `rsi.benchmark()` fits each method once on the development tasks, runs it on every held-out task and seed, and grades the results.

What a method sees (Figure 2). `solve()` receives the task text and a judge, on which it can only call `check(code)`. A check returns whether the program ran, its error, the result of each check, and the causes of the failed ones. It never returns the grade: whether the final program matches an independent reference. The benchmark runner computes the grade after `solve()` returns and never passes it back (§5). Learning methods may use development grades to keep or reject a candidate, but no text a model reads contains a grade.

Equal budgets. Every model call goes through a meter, which refuses a call beyond the budget and records which limit stopped the episode. Each episode gets:

- 3 program executions;
- 6 model calls, counting helper calls such as reflections and reviews;
- 12,000 generated tokens.

Learning methods also get a development budget of 48 episodes, 6 proposals, 600 calls and 1,200,000 tokens. Development episodes count against both budgets. Token counts come from the backend when it reports them; otherwise they are estimated as one token per four characters.

Methods. Table 2 lists the 7 methods.

- `SelfDebug` returns only the error of a program that did not run, and stops as soon as a program runs.

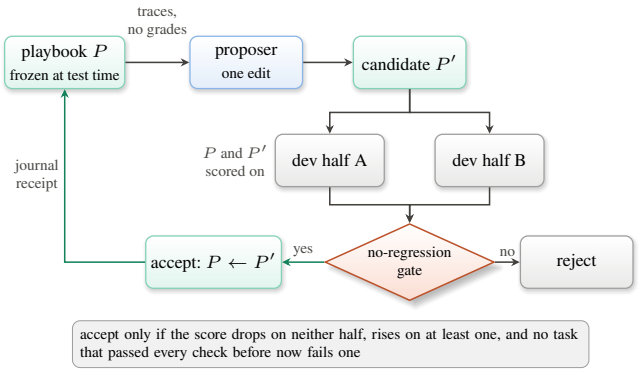


Figure 3: The gated playbook loop: one edit per proposal, scored on two disjoint halves of the development tasks; only the no-regression gate can accept it.

- `CheckFeedback` also returns the failed checks.
- `Reflexion` first asks the model for a reflection of at most five sentences, without code, and keeps the reflections for the rest of the episode.
- In `Team`, a reviewer is asked only when a check of its own family failed. It sees the program, the failed checks, their causes and its family's cards, and answers with a diagnosis rather than code. A program that does not run goes to an interface reviewer.
- `Evolve` rewrites a prompt addendum of at most 1,200 characters from the visible failures of the current best addendum. It keeps a candidate only if it scores strictly higher on the same development tasks: first by the number of valid programs, then by the share of passed checks.

Gated playbook loop. The playbook method is built on the library's `RSILoop` (Figure 3). A playbook is a list of short entries on three editable surfaces: tool usage, retrieval and experience. The proposer sees development traces only. For each failed attempt it sees the program, the failed checks and their causes, but never the grade. One proposal changes one entry. The candidate and the current playbook are both evaluated on two disjoint halves of the development tasks, split by algorithm. The candidate is accepted only if all of the following hold:

- its score does not drop on either half;
- its score rises on at least one half;
- no task that passed every check before fails one under the candidate.

The last condition is the *no-regression* rule. The stricter *all-pass* rule vetoes any candidate while a single task still fails, which blocks learning for a weak model. Whitespace-only edits and candidates that were already tried are rejected. Every proposal, evaluation and decision is appended to a journal whose receipts carry evidence hashes. Timestamps may not move backward, and a frozen playbook cannot change.

4.1 Skill arms

Two switches separate the parts of the skills. `library` adds the interface text and allows `import fin_skills`;

Method	At test time	Learned on development tasks	In the spirit of
`NoImprove`	one program, no feedback	–	baseline
`SelfDebug`	retry with the error only; stop once the program runs	–	Chen et al. (2024)
`CheckFeedback`	retry with the failed checks, their causes and cards	–	verifier feedback
`Reflexion`	as `CheckFeedback`, plus a written reflection before each retry	–	Shinn et al. (2023)
`PlaybookLearning`	as `CheckFeedback`, with a frozen playbook in the prompt	playbook entries, through the gated loop	Zhao et al. (2024); Suzgun et al. (2026)
`Team`	a writer and one reviewer per rule family, from the same model	–	multi-role review
`Evolve`	as `CheckFeedback`, with an evolved prompt addendum	the addendum, over 3 generations on 4 tasks	Agrawal et al. (2026)

Table 2: The 7 self-improvement methods. All share the prompt, the judge and the budget; each keeps the mechanism it is known for. They are not reproductions of the original systems.

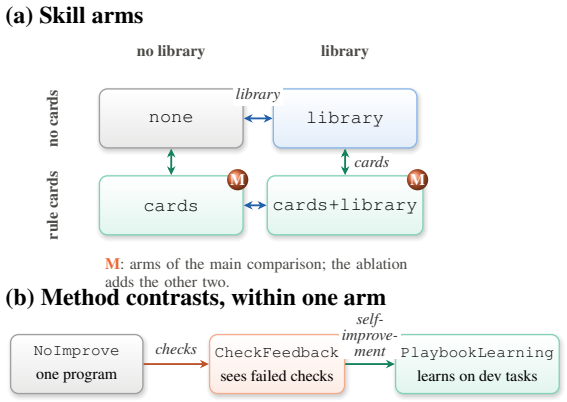


Figure 4: (a) The four skill arms; arrows mark the comparisons that measure the library and the rule cards. (b) Within one arm, `NoImprove` versus `CheckFeedback` measures the checks and `CheckFeedback` versus `PlaybookLearning` measures self-improvement.

`cards` adds the task's rule cards to the prompt and to the check feedback. Together they define 4 arms (Figure 4a).

Every arm keeps the task definition: the rule, the timing convention and the form of the split factors. These fix the correct answer and are not skills. Every arm also sees the failed checks and their causes. Within one arm, `NoImprove` versus `CheckFeedback` measures the checks, and `CheckFeedback` versus `PlaybookLearning` measures self-improvement. For one method, comparing arms measures the skills.

Each arm sees only its own material. Without the library, the worker refuses `import fin_skills`, and cards are rendered without library calls. The development traces given to a proposer follow the same arm.

5 Task suite and protocol

The suite has 34 tasks: 17 algorithms, each with two variants (Appendix A). A task states a rule in prose with its parameters. The model must implement the rule as `solve(prices, split_factors)`, returning a date-by-asset frame. For portfolio and signal tasks the frame holds positions. For volatility, risk and forecast tasks it holds the value known at the previous close. Each task can be solved either by hand or by one library call (`positions()` or `rolling()`).

Data. The inputs are synthetic four-asset panels of 128 business days with seeded Gaussian returns. Raw quotes are adjusted closes divided by cumulative split factors. A panel has four splits at fixed rows: 2-for-1, 3-for-1, 3-for-2 and a 1-for-2 reverse split. The 4 tasks carried over from our earlier study keep 96 rows, and with them only the first two splits, so they reproduce that study's inputs exactly. Variant a gives cumulative factors in alphabetical column order. Variant b gives event ratios, a shuffled column order and different parameters. Every task runs on 3 data seeds (Figure 6a), and held-out tasks also run on one real panel (below). Together, the tasks contain the failures we observed in earlier agent-written programs:

- cumulative factors applied again, or event ratios used as cumulative factors;
- positions decided on the close they trade over;
- warm-up rows computed on a partial window or left missing;
- the missing first return of `pct_change()`;
- columns assumed to be in alphabetical order.

Real prices. The real panel holds the as-traded daily closes of Amazon, Shopify, Alphabet (class A) and Tesla for the 128 sessions from 31 March 2022 to 3 October 2022. All four split in that window: 20-for-1, 10-for-1, 20-for-1 and 3-for-1 (Figure 5). Amazon's split falls on row 45, where asset A splits in the synthetic panels. Columns carry the tickers, in split order for variant a and shuffled for variant b, and each variant keeps its factor convention. The closes come from Yahoo Finance, which serves them split-adjusted. We rebuild the as-traded quotes from a split table checked against the companies' announcements, and the rebuilt closes before each split match published figures. Yahoo's terms forbid redistribution, so we

release the fetch code and the file's SHA-256 rather than the prices. The panel has no reverse split and no dividend.

References. Each task has an independent reference written with numpy, which does not call FIN-RSI. On every task and seed, the reference agrees with the one-step library call to within 2.8×10^{-14}. The exception is `min_variance`, which agrees to within 1.4×10^{-6}: the library solves it numerically, and the reference enumerates the supports of the long-only solution exactly. Building these references exposed a precision defect in the library's long-only minimum-variance optimiser. On daily-scale covariances the optimiser stopped early, and weights differed from the exact solution by up to 1.2×10^{-2}. Rescaling the covariance before solving fixed it.

Judge and hidden grader. The judge runs each program in a separate process with a memory limit and a 120-second timeout. In our runs the process is also confined on Linux with Landlock and seccomp, which restrict its file-system and network access. Without library access, the judge refuses `import fin_skills`. It applies the three checks of §3.2, perturbing from rows 32 and 64 for `execution_lag` (Figure 6b). The grader is separate code that never appears in a prompt. A program is *valid* only if it ran, passed every check, and matches the reference within an absolute tolerance of 10^{-8} (10^{-5} for `min_variance`). A program that passes every check but is wrong is *accepted but wrong*; we report this rate because it is the error the checks cannot see.

Before any model was run, the suite had to pass three qualification tests:

- on every task, a correct program (the one-step library call) passes every check and is graded valid, so no check raises a false alarm;
- on every task and seed, the reference agrees with the library;
- on a position task and a table task, a planted version of each failure fails both its check and the grade: trading on the same close, no adjustment, factors cumulated twice (position task only), and missing warm-up rows.

All three also pass on the real panel; the first is run on the held-out tasks, the only ones the real panel is used for.

Development and held-out split. The split is by algorithm, so no held-out algorithm appears during development. The 5 algorithms that a model has already seen go to development: four from our earlier study and one used in model qualification. Of the remaining 12, a seeded draw sends 4 to development. Development then has 9 algorithms (18 tasks). The held-out set has 8 algorithms (16 tasks) and covers all 5 families.

Protocol. The primary outcome is the valid-delivery rate on held-out tasks. Secondary outcomes are the accepted-but-wrong rate, model calls, generated tokens, wall time and library use.

The main comparison runs 7 methods $\times$ 2 arms (`cards`, `cards+library`) $\times$ 16 tasks $\times$ 3 seeds $\times$ 5 models, or 3,360 episodes. The same comparison on the real panel adds 1,120 episodes, which are reported separately and never pooled with the synthetic seeds. The component ablation adds `NoImprove`, `CheckFeedback` and `PlaybookLearning` on the `none` and `library` arms, a further 1,440 episodes.

Intervals are task-clustered bootstrap intervals, since seeds of one task are not independent. Paired comparisons match the task, seed and model, with a Holm correction. The analysis plan is frozen in a protocol file before inference, and every result is reported whatever its direction. [Models: 5 open-weight models from at least four families, chosen by qualification on development tasks; to be named when the protocol is frozen.]

6 Results

Model qualification. [Qualification on two development tasks: for each candidate model, whether its weights load, whether it returns a program, whether the program runs, and the cost of one episode. To be filled in when the qualification job has run.]

Main comparison. [Valid-delivery rate per method and arm on held-out tasks, with task-clustered 95% intervals, for the models whose runs are complete by the submission deadline; the full matrix of 5 models is reported in the full paper.]

7 Limitations

Most episodes use synthetic price panels with a fixed set of pitfalls. The real panel is one window of four stocks, with forward splits only and no dividends. The checks can reject only the failures they were written for, which is why we report the accepted-but-wrong rate. All models are open-weight, and every method runs under one attempt budget.

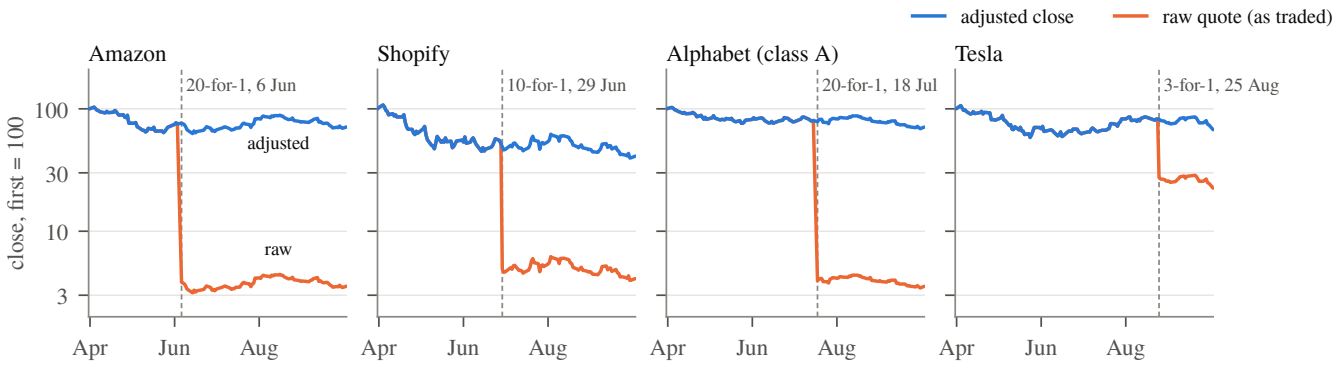


Figure 5: The real-price panel: raw quotes (as traded) and adjusted closes, each indexed to 100 at the first close, on a log scale. A program that ignores the split factors sees a one-day loss of 67% to 95%.

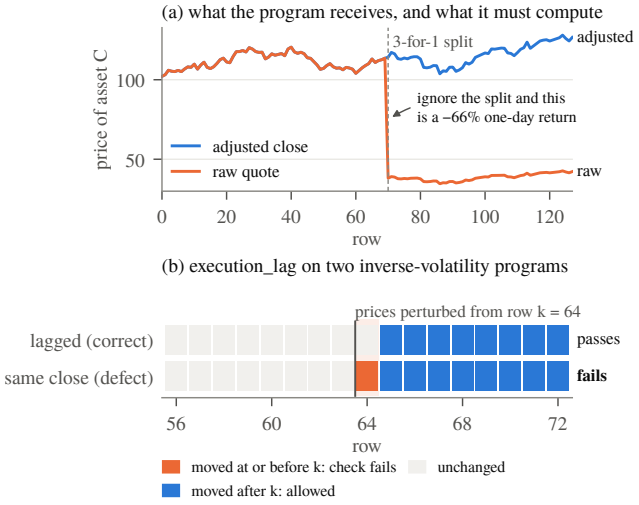


Figure 6: Task data and one check, computed by the library on the seed-11 panel. (a) The program receives raw quotes and split factors; raw and adjusted prices coincide before the split. (b) Prices are perturbed from row $k = 64$ as the guard perturbs them. The library's lagged positions first move at row 65. The same positions read on the close they trade over move at row 64, and `execution_lag` fails.

References

Agent Skills. 2026. Agent skills specification. https://agentskills.io/specification. Accessed 2026-11-04.

Lakshya A Agrawal, Shangyin Tan, Dilara Soylu, Noah Ziems, Rishi Khare, Krista Opsahl-Ong, Arnav Singhvi, Herumb Shandilya, Michael J Ryan, Meng Jiang, Christopher Potts, Koushik Sen, et al. 2026. GEPA: Reflective prompt evolution can outperform reinforcement learning. In *International Conference on Learning Representations (ICLR)*.

David H Bailey, Jonathan Borwein, Marcos López de Prado, and Qiji Jim Zhu. 2017. The probability of backtest overfitting. *Journal of Computational Finance*, 20(4):39–69.

Antoine Bigeard, Langston Nashold, Rayan Krishnan, and Shirley Wu. 2025. Finance agent benchmark: Benchmarking llms on real-world financial research tasks. *arXiv preprint arXiv:2508.00828*.

Xinyun Chen, Maxwell Lin, Nathanael Schärli, and Denny Zhou. 2024. Teaching large language models to self-debug. In *International Conference on Learning Representations (ICLR)*.

Yanxu Chen, Zijun Yao, Yantao Liu, Amy Xin, Jin Ye, Jianing Yu, Lei Hou, and Juanzi Li. 2025. StockBench: Can LLM agents trade stocks profitably in real-world markets? *arXiv preprint arXiv:2510.02209*.

Zhibin Gou, Zhihong Shao, Yeyun Gong, Yelong Shen, Yujiu Yang, Nan Duan, and Weizhu Chen. 2024. CRITIC: Large language models can self-correct with tool-interactive critiquing. In *International Conference on Learning Representations (ICLR)*.

Songqiao Han, Xiyang Hu, Hailiang Huang, Minqi Jiang, and Yue Zhao. 2022. ADBench: Anomaly detection benchmark. In *Advances in Neural Information Processing Systems 35 (NeurIPS 2022), Datasets and Benchmarks Track*.

Jie Huang, Xinyun Chen, Swaroop Mishra, Huaixiu Steven Zheng, Adams Wei Yu, Xinying Song, and Denny Zhou. 2024. Large language models cannot self-correct reasoning yet. In *International Conference on Learning Representations (ICLR)*.

Marcos López de Prado. 2018. *Advances in Financial Machine Learning*. John Wiley & Sons.

Aman Madaan, Niket Tandon, Prakhar Gupta, Skyler Hallinan, Luyu Gao, Sarah Wiegreffe, Uri Alon, Nouha Dziri, Shrimai Prabhumoye, Yiming Yang, Shashank Gupta, Bodhisattwa Prasad Majumder, Katherine Hermann, Sean Welleck, Amir Yazdanbakhsh, and Peter Clark. 2023. Self-refine: Iterative refinement with self-feedback. In *Advances in Neural Information Processing Systems 36 (NeurIPS 2023)*.

Model Context Protocol. 2026. Model context protocol specification, revision 2026-07-28. https://modelcontextprotocol.io/specification.

Marco Tulio Ribeiro, Tongshuang Wu, Carlos Guestrin, and Sameer Singh. 2020. Beyond accuracy: Behavioral testing of NLP models with CheckList. In *Proceedings of the 58th*

Annual Meeting of the Association for Computational Linguistics (ACL), pages 4902–4912.

Sergio Segura, Gordon Fraser, Ana B. Sánchez, and Antonio Ruiz-Cortés. 2016. A survey on metamorphic testing. *IEEE Transactions on Software Engineering*, 42:805–824.

Noah Shinn, Federico Cassano, Ashwin Gopinath, Karthik Narasimhan, and Shunyu Yao. 2023. Reflexion: Language agents with verbal reinforcement learning. In *Advances in Neural Information Processing Systems 36 (NeurIPS 2023)*.

Mirac Suzgun, Mert Yuksekgonul, Federico Bianchi, Dan Jurafsky, and James Zou. 2026. Dynamic cheatsheet: Test-time learning with adaptive memory. In *Proceedings of the 19th Conference of the European Chapter of the Association for Computational Linguistics (Volume 1: Long Papers)*.

Guanzhi Wang, Yuqi Xie, Yunfan Jiang, Ajay Mandlekar, Chaowei Xiao, Yuke Zhu, Linxi Fan, and Anima Anandkumar. 2024. Voyager: An open-ended embodied agent with large language models. *Transactions on Machine Learning Research*.

Qianqian Xie, Weiguang Han, Zhengyu Chen, Ruoyu Xiang, Xiao Zhang, Yueru He, Mengxi Xiao, Dong Li, Yongfu Dai, Duanyu Feng, Yijing Xu, et al. 2024. FinBen: A holistic financial benchmark for large language models. In *Advances in Neural Information Processing Systems 37 (NeurIPS 2024), Datasets and Benchmarks Track*.

Wentao Zhang, Lingxuan Zhao, Haochong Xia, Shuo Sun, Jiaze Sun, Molei Qin, Xinyi Li, Yuqing Zhao, Yilei Zhao, Xinyu Cai, Longtao Zheng, Xinrun Wang, and Bo An. 2024. A multimodal foundation agent for financial trading: Tool-augmented, diversified, and generalist. *arXiv preprint arXiv:2402.18485*.

Andrew Zhao, Daniel Huang, Quentin Xu, Matthieu Lin, Yong-Jin Liu, and Gao Huang. 2024. ExpeL: LLM agents are experiential learners. In *Proceedings of the AAAI Conference on Artificial Intelligence*, volume 38, pages 19632–19642.

Yue Zhao, Zain Nasrullah, and Zheng Li. 2019. PyOD: A Python toolbox for scalable outlier detection. *Journal of Machine Learning Research (JMLR MLOSS)*, 20(96):1–7.

A Task suite

Algorithm	Split	Library call
portfolio		
`cross_sectional_momentum`	dev	`positions()`
`hrp`	heldout	`positions()`
`inverse_volatility`	dev	`positions()`
`min_variance`	heldout	`positions()`
signal		
`bollinger_reversion`	dev	`positions()`
`donchian_breakout`	heldout	`positions()`
`ma_crossover`	dev	`positions()`
`macd`	dev	`positions()`
`momentum`	dev	`positions()`
`rsi_reversion`	heldout	`positions()`
`vol_target_momentum`	dev	`positions()`
volatility		
`ewma_volatility`	heldout	`rolling()`
`historical_volatility`	dev	`rolling()`
risk		
`historical_var_es`	heldout	`rolling()`
`normal_var_es`	heldout	`rolling()`
forecast		
`drift`	heldout	`rolling()`
`mean`	dev	`rolling()`

Table 3: The 17 algorithms of the task suite, with their split and the one-step library call that solves them. Each algorithm has two tasks (variants a and b).